

CISTR

Tema 1. Programación Concurrente y Sistemas en Tiempo Real

José Manuel González Cava
jgonzalc@ull.edu.es

Conceptos

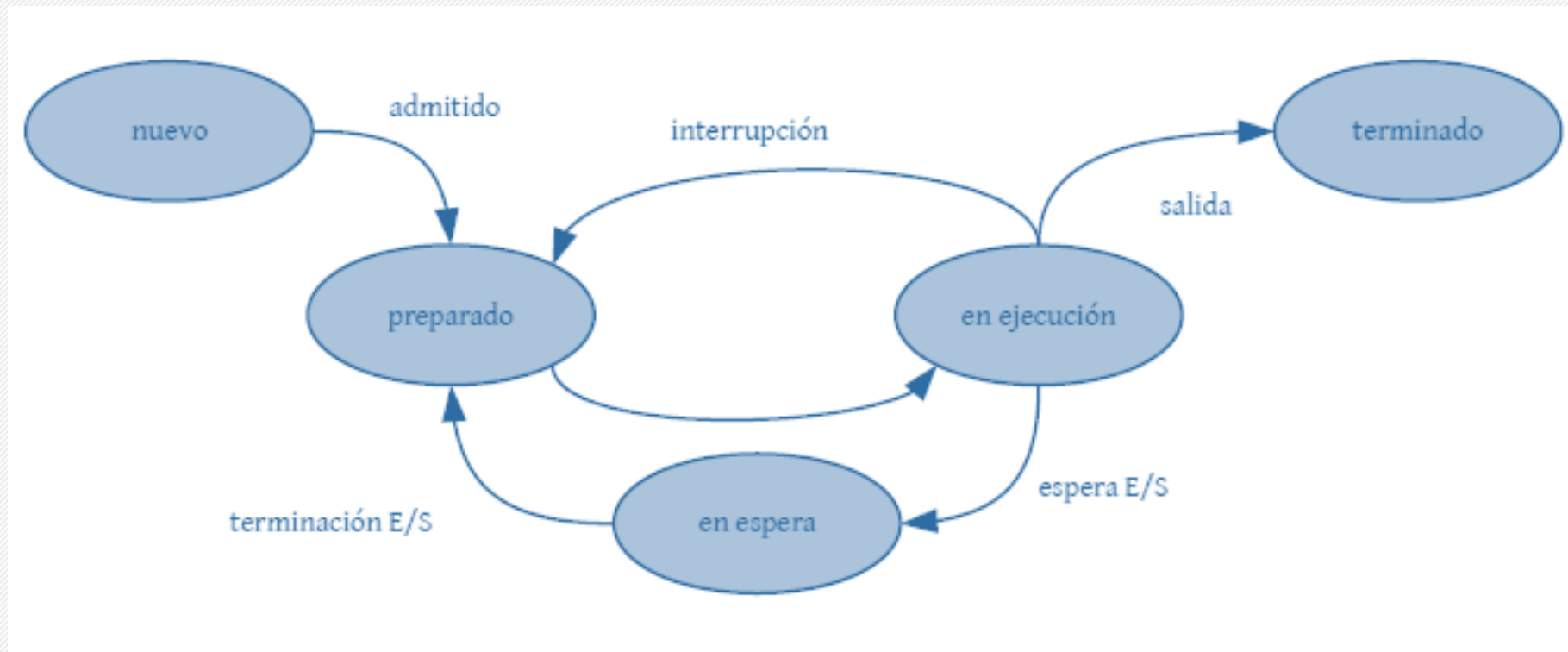
La **programación concurrente** está vinculada al concepto de sistema operativo multiproceso, donde los procesos comparten todo tipo de recursos: desde la CPU hasta cualquier periférico.

Un **proceso** se puede entender como un programa en ejecución. Está formado por:

- Pila: almacena datos temporales (variables locales, direcciones de retorno o parámetros de funciones).
- Datos: variables globales.
- Memoria.

Conceptos

Diagrama general de **estados** de un proceso:

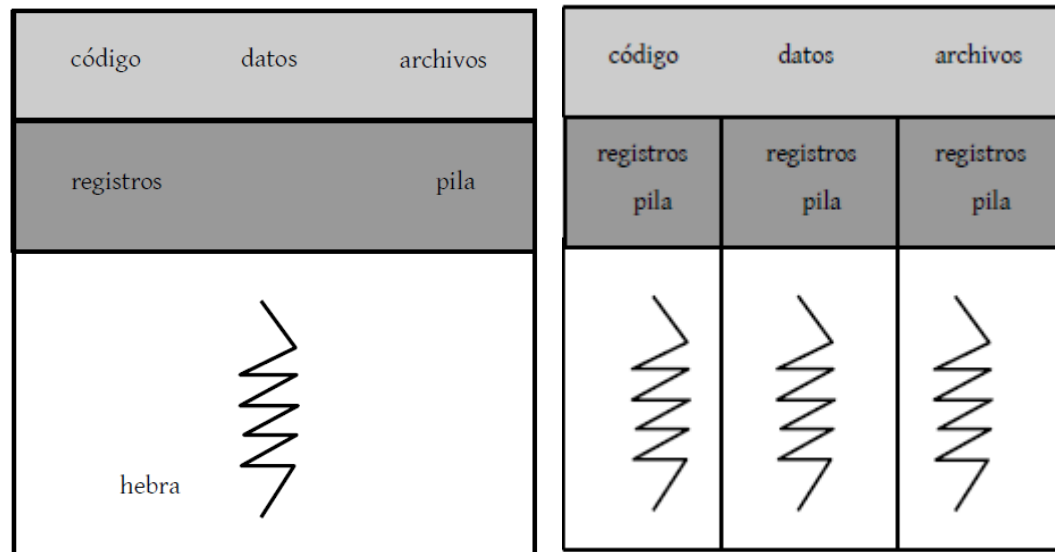


Conceptos

La **multiprogramación** es la posibilidad de manejar distintos hilos de ejecución de manera simultánea con el objetivo de paralelizar el código e incrementar el rendimiento.

Un **hilo** es la unidad básica de utilización del procesador y está compuesta por:

- ID hilo.
- Contador de programa.
- Registros.
- Pila



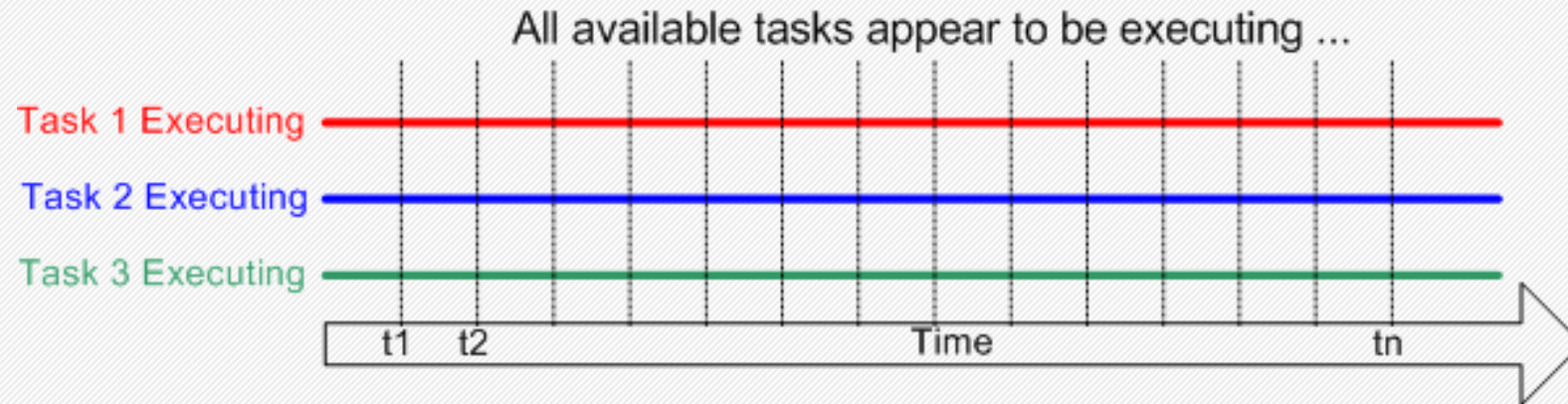
Conceptos

Ventajas de la programación multihilo:

- ✓ *Capacidad de respuesta*: ejecuta distintas funciones de forma simultánea.
- ✓ *Compartición de recursos*: varios hilos manejan el mismo espacio de direcciones.
- ✓ *Eficacia*: manejo más óptimo frente a procesos.

Un único procesador es capaz de ejecutar múltiples hilos de manera “simultánea”. Si se habla de **paralelismo**, entonces existen múltiples núcleos físicos ejecutando códigos en paralelo.

Conceptos



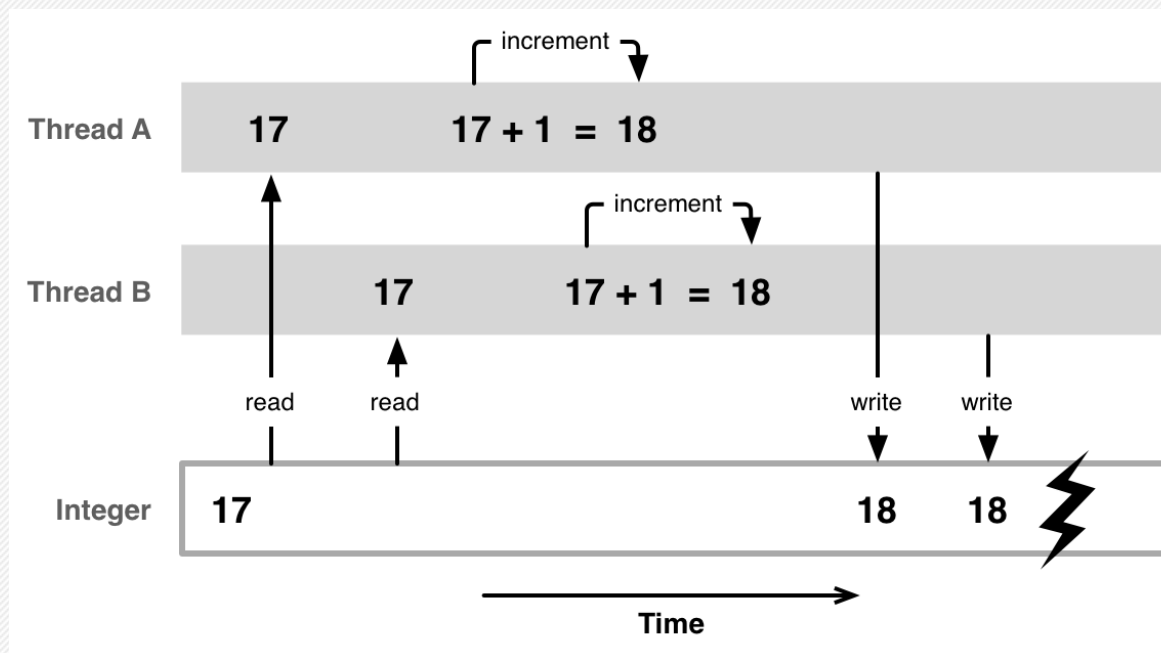
Fundamentos de programación concurrente

Un **proceso cooperativo** es aquél que puede verse afectado por otros procesos en ejecución (intercambio de datos, archivos o uso del mismo código). El acceso concurrente a datos compartidos puede generar **inconsistencias**.

- **Condiciones de carrera.**
- **Sección crítica.**

Fundamentos de programación concurrente

- **Condiciones de carrera:** comportamiento del software en el que el resultado de la operación depende del orden de las instrucciones. El comportamiento del programa es imprevisible.



Fundamentos de programación concurrente

- **Sección crítica:** Segmento de código dentro de un proceso en la que se accede a un recurso compartido. Debe establecerse un mecanismo de exclusión.
 - **Sección de entrada:** se solicita acceso a la sección crítica. Mecanismo de acceso exclusivo (sincronización).
 - **Sección crítica:** modificación del recurso compartido.
 - **Sección de salida:** se notifica la salida de la sección crítica mediante la liberación del mecanismo de sincronización.
 - **Sección restante:** ejecución resto del código.



Fundamentos de programación concurrente

- **Sección crítica:** cualquier solución debe satisfacer los siguientes requisitos:
 - 1) **Exclusión mutua:** si un proceso está en la sección crítica, ningún otro proceso puede ejecutar su sección crítica asociada.
 - 2) **Progreso:** se decidirá qué proceso entra en la sección crítica en un tiempo limitado.
 - 3) **Espera limitada:** se limita la espera de los procesos para el acceso a sus secciones críticas.

Mecanismos básicos de sincronización

Semáforos

Mecanismo de sincronización que encapsula una variable entera, únicamente accesible a través de las operaciones *wait* y *signal*.

- Los semáforos deben inicializarse con un valor NO negativo.
- La operación *wait* decrementa el valor del semáforo. Si este valor es negativo, entonces el proceso se bloquea.
- Un valor negativo representa el número de procesos bloqueados.



Mecanismos básicos de sincronización

Semáforos

Mecanismo de sincronización que encapsula una variable entera, únicamente accesible a través de las operaciones *wait* y *signal*.

- La operación *signal* incrementa el valor del semáforo. Si el valor no era positivo, entonces se desbloquea un proceso previamente bloqueado por *wait*.
- Un valor positivo representa el número de procesos que pueden hacer *wait* sin quedarse bloqueados.
- Si el valor del semáforo es 0, implica que no hay procesos en espera, pero una operación *wait* provocará el bloqueo.



Mecanismos básicos de sincronización

Semáforos

Hay dos tipos:

1. *Semáforos binarios (cerrojo mutex)*. Toma valores de 0 ó 1. Se emplea como mecanismo de acceso exclusivo a la sección crítica.
2. *Semáforo contador*. Su valor puede variar sin restricciones. Gestiona el acceso concurrente a distintas instancias de un recurso.

Mecanismos básicos de sincronización

Semáforos

Características:

- ✓ Simplicidad en su uso.
- ✓ Soluciones limpias y legibles.
- ✓ Alta portabilidad a distintas plataformas y SO.
- ✗ Requieren de espera activa. Se evita habilitando el bloqueo del proceso.
- ✗ Inanición (starvation). Se priorizan determinados procesos frente a otros que puedan quedar en espera indefinida.

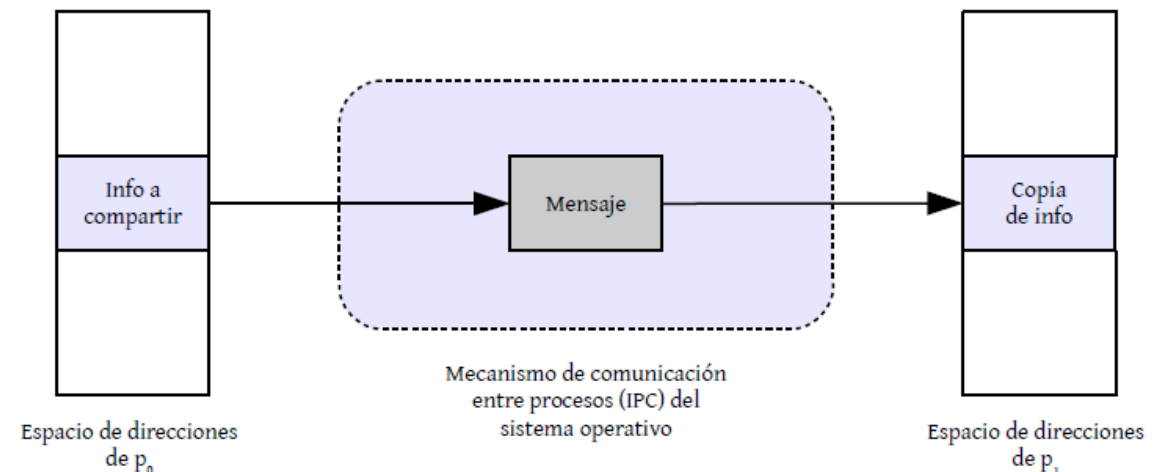
Mecanismos básicos de sincronización

Paso de mensajes

Mecanismo de sincronización que permite comunicación entre procesos sin tener que recurrir a variables de memoria compartida. Se trata de una abstracción del concepto de comunicación.

Está basado en el uso de dos primitivas:

- **Send:** permite el envío de información a un proceso.
- **Receive:** permite la recepción de información por parte de un proceso.



Mecanismos básicos de sincronización

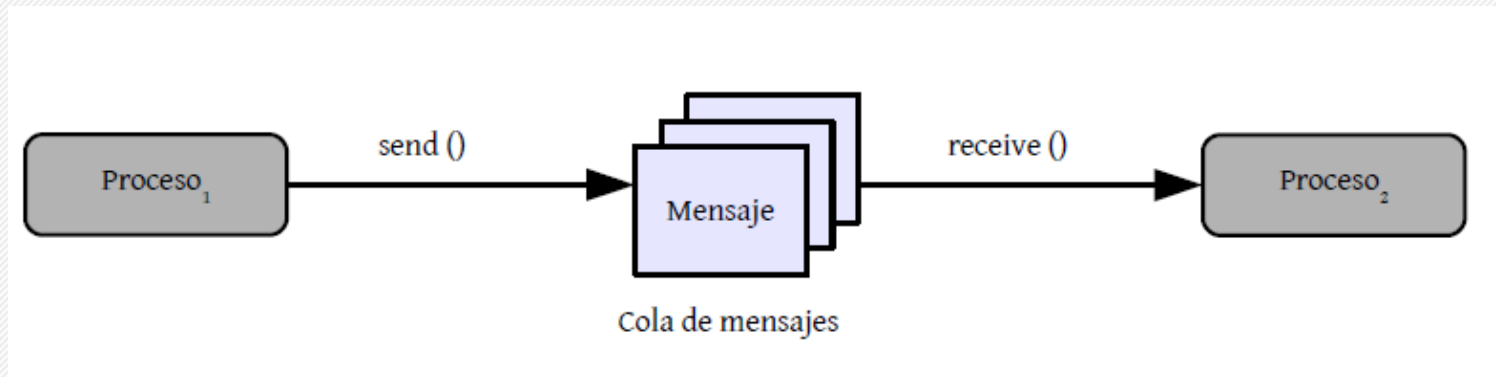
Paso de mensajes

- i. El proceso emisor genera un mensaje como un bloque de información.
- ii. El emisor pide al SO que entregue un mensaje en el espacio de direcciones del proceso receptor.
- iii. Se recupera el mensaje del espacio de direcciones del emisor.
- iv. El mensaje se coloca en un buffer del SO.
- v. Se copia el mensaje del buffer en el espacio de direcciones del receptor.

Mecanismos básicos de sincronización

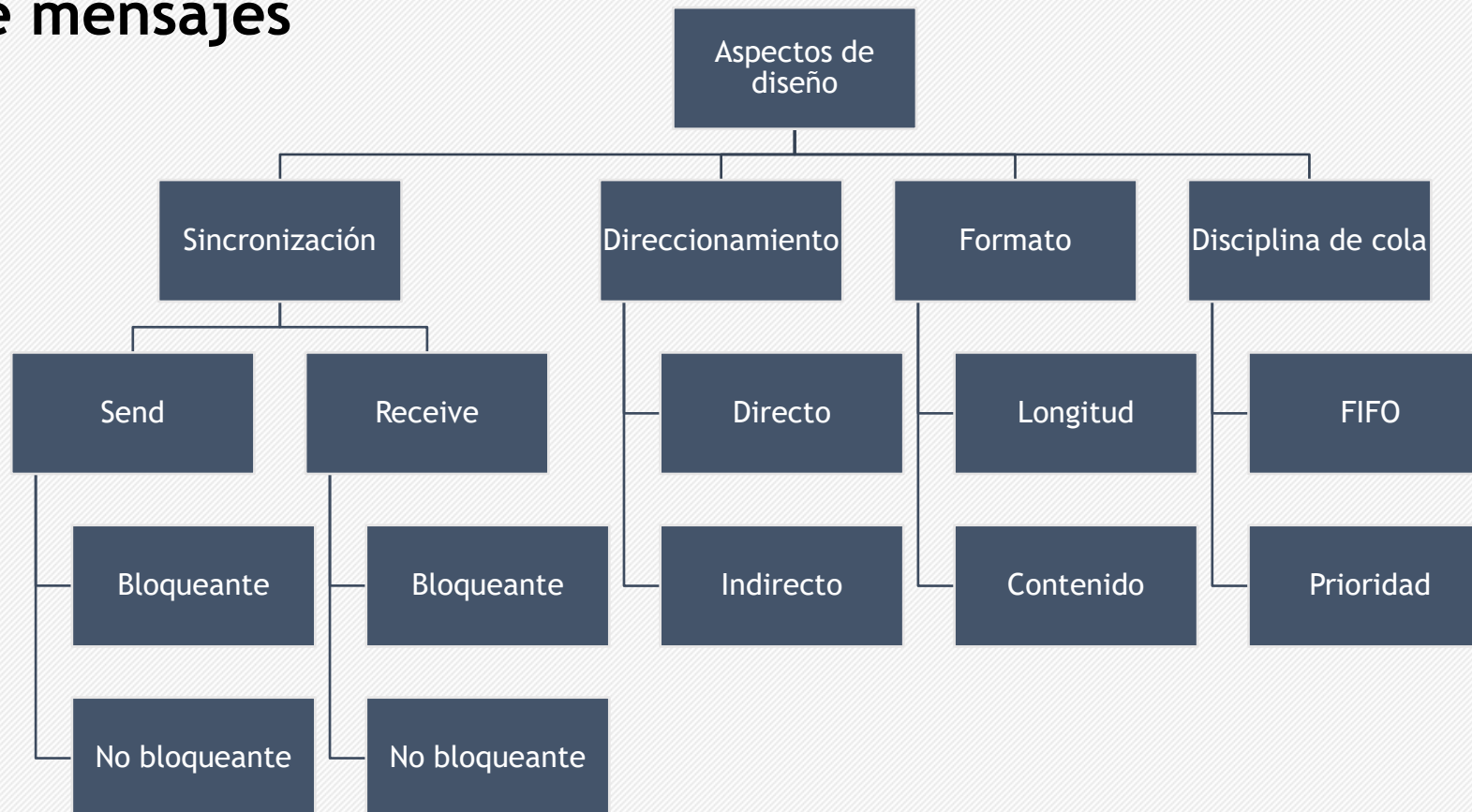
Paso de mensajes

El SO puede almacenar los mensajes de entrada en un **buzón** o **cola de mensajes**.



Mecanismos básicos de sincronización

Paso de mensajes



Mecanismos básicos de sincronización

Monitores

- X El uso adecuado de los semáforos depende del programador.
- X Los semáforos y colas de mensaje tiene baja flexibilidad.

Los **monitores** son soluciones de alto nivel basada en el concepto de encapsulación de un recurso compartido.

Gestionan el acceso al recurso mediante métodos sincronizados que garantizan que haya únicamente un proceso activo.

Mecanismos básicos de sincronización

Monitores

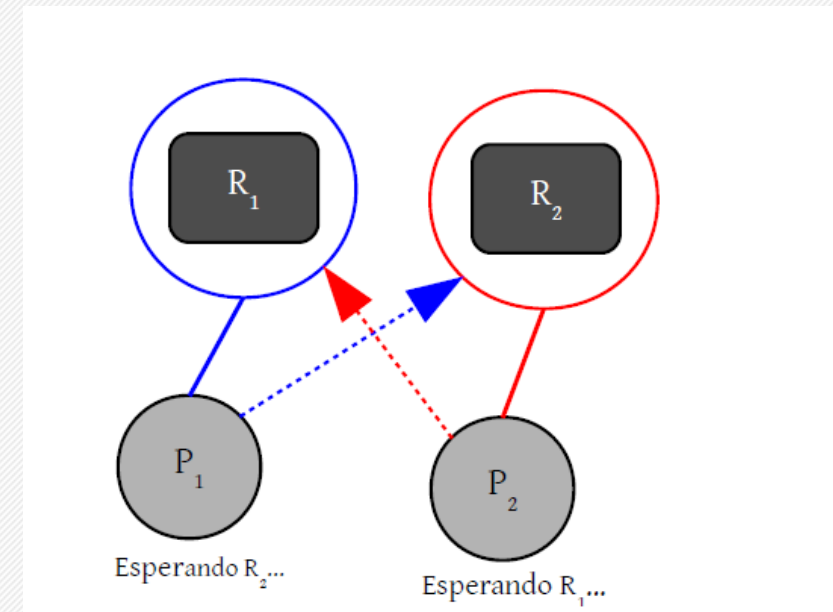
Está formado por cuatro componentes:

1. **Inicialización:** código a ejecutar en la creación del monitor.
2. **Datos privados:** procedimientos privados a usar desde dentro del monitor (no visibles desde fuera).
3. **Métodos del monitor:** procedimientos que pueden ser llamados desde fuera del monitor.
4. **Cola de entrada:** contiene a los hilos que han llamado a algún método y están a la espera de acceso.

Deadlocks

Deadlocks / Interbloqueo

Un conjunto de procesos estará en **deadlock** cuando todos los procesos del conjunto estén esperando a que se produzca un suceso que sólo puede darse como resultado de la actividad de otro proceso del conjunto.



Deadlocks

Deadlocks / Interbloqueo

Los interbloqueos pueden surgir si se dan de forma simultánea las cuatro condiciones de Coffman:

1. **Exclusión mutua:** al menos un recurso está en modo no compartido. Un proceso debe esperar a la liberación del recurso.
2. **Retención y espera:** proceso reteniendo al menos un recurso y a la espera para adquirir otros recursos.
3. **No apropiación:** un recurso sólo puede ser liberado por el proceso que lo retiene después de finalizar la tarea.
4. **Espera circular:** hay un conjunto de procesos en competición por un recurso

Deadlocks

Deadlocks / Interbloqueo

Los interbloqueos se pueden tratar de 3 formas:

1. Impedir o evitar los interbloqueos, asegurando que al menos una de las condiciones de Coffman no se cumpla.
 - Exclusión mutua sólo si es estrictamente necesario.
 - Política de asignación de recursos.
 - Política de desalojo de recursos.
 - Relación de orden de todos los tipos de recursos.

Deadlocks

Deadlocks / Interbloqueo

Los interbloqueos se pueden tratar de 3 formas:

2. Detectar y recuperarse del interbloqueo, implementando un mecanismo de identificación y política de resolución.
3. Ignorar los interbloqueos, asumiendo que nunca se van a dar (algoritmo de la avestruz). Es el programador quien debe lidiar con ellos.

Fundamentos de Tiempo Real

Sistema en tiempo real (STR / RTS)

“Cualquier sistema en el que el tiempo en el que se produce la salida es significativo (...).”

- Se debe realizar una determinada funcionalidad atendiendo a una serie de restricciones temporales esenciales para la aplicación.
- Se debe asegurar la respuesta en un intervalo acotado y perfectamente definido para garantizar el funcionamiento del sistema.
- El concepto de **temporalidad** es clave frente a otros sistemas.

Fundamentos de Tiempo Real

Sistema en tiempo real (STR / RTS)

Proceso de compra de entradas para un concierto a través de una web.

Airbag de un vehículo.

Seta de emergencia de un sistema industrial.

Fundamentos de Tiempo Real

Sistema en tiempo real (STR / RTS)

Pueden clasificarse en:

- **Estrictos (Hard RT):** Siempre debe cumplir con los requisitos temporales.
- **No estrictos (Soft RT):** aunque el tiempo de respuesta sigue siendo importante, no es absolutamente necesario que se produzca antes de un deadline.

Fundamentos de Tiempo Real

Mecanismos de planificación

Relojes

Establecen una referencia temporal.

Retardos

Capacidad de un proceso o hilo para suspenderse durante un determinado instante de tiempo. Pueden ser:

- Relativos.
- Absolutos.

Fundamentos de Tiempo Real

Esquemas de planificación/scheduling

Mecanismo que garantiza que los distintos procesos ejecutan sus tareas en el plazo cumpliendo con las restricciones definidas.

Está formado por:

- Algoritmo para **ordenar** el uso de los recursos del sistema (CPU).
- Mecanismo para **predicción** del comportamiento del sistema en base al algoritmo anterior. Permite anticipar si se cumplirán los requisitos temporales.

Fundamentos de Tiempo Real

Esquemas de planificación/scheduling

- **Tiempo máximo de ejecución (C)**, tiempo de ejecución de un proceso en el peor de los casos.
- **Interferencia (I)**, tiempo que acumula un proceso como consecuencia de la interrupción de otros.
- **Bloqueo (B)**, tiempo de bloqueo del proceso en el peor caso.
- **Deadline (D)**, tiempo límite de ejecución de un proceso.
- **Tiempo de respuesta (R)**, tiempo de respuesta en el peor caso.
- **Periodo (T)**, tiempo mínimo entre dos ejecuciones.
- **Prioridad (P)**, importancia del proceso.

Fundamentos de Tiempo Real

Planificación basada en procesos

¿Cuál es el proceso a ejecutar en cada momento?

Dos estrategias:

1) Fixed Priority Scheduling (FPS)

- Consiste en asignar una prioridad fija y estática a un proceso.
- Una vez se calcula la prioridad de todos los procesos, se establece el orden de ejecución.
- Surge la *apropiación*, donde un proceso de alta prioridad puede desalojar a un proceso de baja prioridad.
- Es uno de los más utilizados en la actualidad.

Fundamentos de Tiempo Real

Planificación basada en procesos

¿Cuál es el proceso a ejecutar en cada momento?

Dos estrategias:

2) Earliest Deadline First (EDF)

- Planificación por límite más cercano.
- El próximo proceso a ejecutar es que el tenga menor deadline absoluto.

Fundamentos de Tiempo Real

Características de un planificador

El planificador determina qué tarea debe ejecutarse en CPU. Para ello, debe gestionar:

1. Un sistema de **asignación de prioridades**.
 - Las prioridades están asociadas a requisitos temporales.
 - Rate monotonic: los procesos con menores periodos tienen prioridad más alta.
2. Modelo de análisis del tiempo de respuesta.
 - *Factor de utilización.*
 - *Esquema gráfico.*
 - *Cálculo de tiempo de respuesta.*

Fundamentos de Tiempo Real

Características de un planificador

Factor de utilización.

- En un esquema FPS, permite garantizar la planificación de los procesos.

$$\sum_{i=1}^N \left(\frac{C_i}{T_i} \right) \leq N(2^{\frac{1}{N}} - 1)$$

- C: tiempo máximo de ejecución.
- T: periodo.
- N: número de procesos

Fundamentos de Tiempo Real

Características de un planificador

Factor de utilización.

Ejercicio: Calcular el factor de utilización para los procesos a, b y c y determina si el sistema es planificable.

Proceso	T	C	P	U
A	100	20	1	?
B	40	15	2	?
C	30	5	3	?

Solución: $U_A = 0.2$; $U_B = 0.38$; $U_C = 0.17$ $U_T = 0.75 < 0.78$ **Planificable**

Fundamentos de Tiempo Real

Características de un planificador

Cálculo de tiempo de respuesta

- Se calcula el tiempo de respuesta de los procesos (R) analíticamente.
- Un sistema es planificable si:

$$R_i \leq D_i \forall i \in \{1, 2, \dots, n\}$$

Fundamentos de Tiempo Real

Características de un planificador

Cálculo de tiempo de respuesta

1. Se evalúa el proceso de mayor prioridad.

$$R_i = C_i$$

2. Para cualquier otro proceso:

$$R_i = C_i + I_i$$

$$R_i = C_i + \sum_{j \in hp(i)} (\lceil \frac{R_i}{T_j} \rceil * C_j)$$

$$w_i^n = C_i + \sum_{j \in hp(i)} (\lceil \frac{w_i^{n-1}}{T_j} \rceil * C_j)$$

Fundamentos de Tiempo Real

Características de un planificador

Cálculo de tiempo de respuesta

3. Se parará de iterar cuando:

a) $w_i^{n-1} = w_i^n$

b) Si $w_i^n > D_i$, por lo que el sistema no será planificable.

Fundamentos de Tiempo Real

Características de un planificador

Cálculo de tiempo de respuesta

Ejercicio: Calcular el tiempo de respuesta de los siguientes procesos y comprueba si es planificable

Proceso	T	C	P
A	100	20	1
B	40	15	2
C	30	5	3

Solución: $R_C = 5 (< 20)$; $R_B = 20 (< 40)$; $R_A = 60 (< 100)$ **Planificable**

Herramientas para STR

1. Sistemas Operativos en Tiempo Real

- **Funcionalidad en tiempo real.**
- **Latencia de interrupción mínima**, para minimizar el tiempo transcurrido desde que se recibe una interrupción hasta que se ejecuta en CPU.
- **Planificación de tareas apropiativo**, asegurando la ejecución de tareas de altas prioridades.
- **Implementación de algoritmos de planificación**, asegurando que se cumplen con los requisitos temporales.

Herramientas para STR

1. Sistemas Operativos en Tiempo Real

- Manejo de concurrencia.
- Secciones críticas cortas, para evitar tiempos muertos largos de tareas en espera.
- Eficiencia en la programación.
- Robustez del software
- Interacción HW.

Herramientas para STR

1. Sistemas Operativos en Tiempo Real

Algunos ejemplos de RTOS:



Herramientas para STR

2. Lenguajes de programación

- Las características de un sistema en tiempo real vienen proporcionadas por RTOS. Sin embargo, hay lenguajes de programación que facilitan estas tareas (gestión de tiempo, sincronización, comunicación,...).
- Esto permite exportar el código entre diferentes RTOS de forma más sencilla.
- Entre las características de este tipo de lenguajes destaca: soporte de concurrencia, mecanismos de acceso a I/O, gestión de tiempo, soporte de planificación y manejo de excepciones.

Herramientas para STR

2. Lenguajes de programación

- C/Real-Time Posix

C es un lenguaje de programación de propósito general para programación de bajo nivel, código de SO y sistemas empuotrados.

La extensión POSIX (Portable Operating System Interface) es una familia de estándares con el objetivo de manejar la portabilidad entre distintos sistemas operativos. Dota a C de propiedades para el manejo de atributos de lenguajes de programación en tiempo real.

Herramientas para STR

2. Lenguajes de programación

- Ada

Ada fue desarrollado para el Departamento de Defensa de US. Su uso se centra en aplicaciones críticas.

Se trata de un lenguaje ampliamente usado con soporte de concurrencia y sistemas en tiempo real.

Es un lenguaje orientado a objetos (POO).



Herramientas para STR

2. Lenguajes de programación

- Java

Lenguaje de programación Orientado a Objetos con sintaxis similar a C/C++.

Incluye la implementación de hilos como POSIX o Ada para la programación de concurrencia.

Permite la gestión de tiempo mediante interrupciones.



CISTR

Tema 1. Programación Concurrente y Sistemas en Tiempo Real

José Manuel González Cava
jgonzalc@ull.edu.es